



CITS5017 Deep Learning Semester 2, 2026

Project 2

Assessed, worth 20%. Due: 11:59pm, Friday, 16th October 2026

1. Outline

The Australian Bureau of Meteorology (<http://www.bom.gov.au/index.php>) has been collecting climate data of Australia for more than 100 years (see <https://reg.bom.gov.au/climate/history/temperature/>). There are many weather stations located across Australia. Most of these stations have been recording climate data from 1994 onward and some older stations have climate data even earlier. Apart from the publicly available temperature data, many weather stations also release data for other weather elements, such as rainfall and solar exposure. Unfortunately, these weather elements do not have long historical data that can be used for this project. So, we will focus on using just the monthly average minimum and maximum temperature data recorded at the following five weather stations in Western Australia:

- *Perth Airport* (weather station number 009021), which has temperature data from 1944 January to 2026 August (total: 987 months, i.e., approximately 82 years' data).
- *Jandakot Aero* (weather station number 009172, approximately 21.2km away from *Perth Airport*), having temperature data from 1994 January to 2026 August (total: 392 months, i.e., approximately 32 years' data).
- *Perth Metro* (weather station number 009225, approximately 8.5km away from *Perth Airport*), having temperature data from 1994 onward.
- *Swanbourne* (weather station number 009215, approximately 18.7km away), having temperature data from 1993 onward.
- *Bickley* (weather station number 009240, approximately 18.2km away), having temperature data from 1994 onward.

The data downloaded from the website above have been preprocessed and a pickle file is provided on LMS. It comprises:

- the training temperatures sequences, constructed from the data from the *Perth Airport*, *Jandakot Aero*, and *Perth Metro* stations;
- the validation temperatures sequences, constructed from the 396 months' data from the *Swanbourne* station; and
- the testing temperatures sequences, constructed from the 392 months' data from the *Bickley* station.

Your tasks for the project consist of two parts: (i) implement three algorithms for predicting the minimum and maximum temperatures and compare their performances; (ii) train a variational autoencoder (VAE) to synthesize temperature sequences.

2. Submission

Name your Jupyter Notebook file as **Surname.FirstName-proj2.ipynb** (please replace **Surname** and **FirstName**¹ by your own surname and first name).

¹You can include your middle name if you like. It does not matter. We just want to be able to distinguish all the students' submitted files when we put the whole class's submissions together in one large directory when we mark all the projects.

Submit your Jupyter Notebook file to LMS. You should reserve at least half an hour for the submission procedure just in case you have internet issues. You should see an acknowledgement screen confirming that your submission has completed successfully. You can take a screen shot and keep the image as evidence of your submission. Do **not** send your .ipynb file via email to the Unit Coordinator. Do **not** upload your submission onto Google Drive or OneDrive and then send the link to the Unit Coordinator. **Only files submitted to LMS will be marked.**

NOTE: Do not blindly copy examples that you found on the internet. Many of these examples are often too complex and they do not meet the project specification here. **Significant penalty** will be imposed on your mark if your code is found to be copied from the internet. You should be able to complete this project by following examples in the lectures, textbook, and sample codes from the author Géron.

3. The temperatures sequences

A pickle file named *temperatures-2026.pkl* is supplied on the LMS page of the unit. Download this file and save it in the same directory where you would develop your code. Use the following function to load the data:

```
def load_data(picklefile):
    import pickle
    f = open(picklefile, "rb")
    data = pickle.load(f)
    f.close()
    return data

(train_seq, val_seq, test_seq) = load_data("temperatures-2026.pkl")
```

The output variables `train_seq`, `val_seq`, and `test_seq` from running the code above should be TensorFlow tensors of shapes (1522, 84, 2), (313, 84, 2), and (297, 96, 2), respectively. That is, the sequence lengths for the training and validation sets have been prepared to be 84 months long, but the test set has 96-month long sequences. On the last axis (`axis = 2` or `axis = -1`), component 0 stores the monthly mean minimum temperatures and component 1 stores the monthly mean maximum temperatures. The temperatures sequences in `train_seq` have already been randomly shuffled. So, no further shuffling should be needed. The temperatures sequences in `val_seq` and `test_seq` are not shuffled, as shuffling is not required for the validation and test sets.

4. Tasks for part (i)

- (a) Split `train_seq` to form your input sequences `X_train` and target sequences `y_train`. The `input_length` for `X_train` and the `target_length` for `y_train` should be 72 and 12 (months) respectively². Repeat this process for `val_seq` and `test_seq` to form your `X_val`, `y_val`, `X_test`, and `y_test`. Note that the last 12 months of the sequences in the test set are not used until we come to a later part of the project.

(b) **The `display_temperatures` function**

Implement a function named `display_temperatures` that would take in at least two arguments,

NOTE: If your surname and/or first name contain special characters, such as apostrophes, `'`, etc, you will have to omit them or replace them by underscores or hyphens as these characters have special meanings: apostrophes are considered the same as single quotes; `'` is used to denote directories.

²Note: TensorFlow has similar functions like those from the numpy library. For instance, `val_seq.shape` should output the shape (313, 84, 2) of the TensorFlow tensor `val_seq`. You are suggested not to convert the TensorFlow tensors to numpy arrays as this seemed to cause problems in past years.

x (a tensor containing the *input* sequences) and y (a tensor containing the corresponding *target* sequences), randomly sample 3 combined sequences, and display them in 1×3 (i.e., 1 row and 3 columns) subplots. You will need to have additional arguments for this function (see later parts of the project specification). The function should display both the sequences for the mean minimum and mean maximum temperatures.

Call the function for sequences coming from the training set (x_{train} , y_{train}). Repeat this for the validation set and for the test set.

(c) Implementation of model 1

- Design a two-layered GRU network that would learn the mean minimum and maximum temperature patterns of 72 months and predict the following 12 months' mean minimum and maximum temperatures all in one go. Obviously, you would need a *Dense* layer at the top layer of the network and maybe a *Reshape* layer. For the two GRU layers, up to 16 neurons each should be sufficient.
- Illustrate the architecture of your network. Train it for 100 epochs with early stopping, using the training and validation sets and an appropriate optimizer. You might need to manually tune the hyperparameters of the model. However, do not include the extra code cells that you implement for tuning your model. You should include only the code for the best model that you have found. Also, do not display the training and validation losses, metrics, etc, during training. Instead, illustrate the learning curve graphically after the training has completed. The metric of interest is the *mean absolute error* (MAE).
- Apply the model to the test set, evaluate and display its MAE. Illustrate the predicted 12 months' mean minimum and maximum temperatures for 3 randomly sampled sequences against the ground truth target sequences. You should revise your `display_temperatures` function in (b) above (e.g., by adding in optional argument(s)) so that it can handle this additional task.

This network should work pretty well. The important key is to use an appropriate *activation function* for each GRU layer. Depending on your network architecture and the chosen activation function(s), you may need to do appropriate scaling on the data³.

You should try to modularize your code for the tasks above, i.e., put them in appropriate helper functions so that you can reuse them again later. For example, splitting the data sequences to form x_{test} , y_{test} , etc, setting up the network, training the model, and showing the learning curve can all be put into functions. If you have suitable functions written here, then they can be used for model 2 and model 3 later on.

(d) Implementation of model 2

- Design a similar two-layered GRU network that would learn the mean minimum and maximum temperature patterns of 72 months and predict the following 1 month's mean minimum and maximum temperatures. You can reuse the same architecture for model 1 above and just change the shape of the output layer. You should use the same training set for training this model, except that y_{train} would need to be modified to 1 month long when passed to the model for training and prediction.
- Repeat the same subtasks as for model 1 above.
- The last subtask for model 2 is to get it to predict 12 months' minimum and maximum temperatures using a *for* loop (see a similar example in the lecture note and in Chapter 15 of the textbook). The test set designed in part (a) should be used for this last subtask here. Report the MAE of the 12 months' temperature predictions. Three randomly sampled sequences should be illustrated using the same `display_temperatures` function.

³Note: Think of your input and target sequences as 1D images. Do not apply arbitrary nonsensical feature scaling operations.

(e) **Implementation of model 3**

- Design a two-layered GRU Encoder-Decoder network (see lecture topic 8) to do the 12 months' predictions. You can use model 1 for the Encoder part of the network. Feel free to reduce the numbers of neurons of the GRU layers or use a different learning rate scheduler. **Note:** the hidden state vectors from the two GRU layers should be passed to the Decoder part of the network⁴. In essence, the Encoder learns the temperature patterns in the input 72-month sequences while the Decoder learns how to predict the target 12-month sequences.
- See part (c) above for the similar subtasks required for this model. Note that the correct implementation for this model is: at the testing stage (with the test set), **inputs to the Decoder** should come from the predicted temperatures for the previous months, not from the ground truth.

(f) **Comparing models 2 and 3 for different prediction horizons**

Up to now, the lengths of the input sequences and the target sequences have been fixed to 72 and 12. Obviously, if we increase the target length (often referred to as the *prediction horizon*), the performances of all the two models are expected to drop (i.e., the MAEs would increase). Of interest is how fast do the MAEs increase for these models.

Evaluate the performances of models 2 and 3 above using the following settings on the **test set**: (72, 15), (72, 18), (72, 21), and (72, 24) where, for each tuple, the first element denotes the input length (in months) and the second element denotes the target length (in months).

To accomplish the task above, you will need to bring in additional months' data from the `test_seq` variable. For the last setting (72, 24) setting, all the 96 months' data in the test set will be needed. If your code were designed well in earlier parts of the project, the task here should be quite simple and can be done using a simple loop of function calls. Note that both models should not be retrained.

Illustrate the *MAE* versus *prediction horizon* plot of the two models for comparison. Include the original setting (72, 12) in your plot. Which of these two models deteriorates the fastest? How good/bad are these two models in learning the temperature patterns from one weather station to predict the temperatures in another weather station? Describe your results in a markdown cell.

5. Tasks for part (ii)

This is just a fun part of the project: to investigate if a VAE can synthesize realistic looking temperatures sequences. To simplify the task involved, we will focus on synthesizing sequences for the **maximum temperatures** only.

- The inputs to your VAE should be maximum temperature sequences of 84 months obtained from `train_seq`. Design a suitable *codings* size. Your VAE model can consist of *Dense* layers for both the Encoder and Decoder parts. Alternatively, it can have *Conv1D* layers for the Encoder part and *Conv1DTranspose* layers for the Decoder part. Two layers should be sufficient for each part of the VAE model.
- Train your model with an appropriate optimizer, and optionally a learning rate scheduler for 100 epochs, with early stopping. The `val_seq` (only for the maximum temperatures part) can be used as the validation set. Again, do not show verbose training output. Instead, illustrate the learning curve graphically.
- Illustrate 6 synthesized temperature sequences (in 2×3 subplots) by randomly sampling 6 *codings* vectors.

⁴This means that your Decoder for this model should also have two GRU layers.

- You should insert markdown cells to explain assumptions you made, data preparation, and implementation for the tasks above. Do your temperature sequences look realistic? Give a brief explanation to your results.

6. How to structure your Jupyter Notebook file

Your Jupyter Notebook file should be considered as a document with code. Alongside the code cells, explanation should be included in markdown cells. You should try to structure your Jupyter Notebook file by dividing it into sections and subsections with meaningful headings. **Do not simply copy and paste the project specification to form your markdown cells.**

It is likely that you would have experimental code for yourself during the implementation of the project. These experimental code cells should be removed in the final version; otherwise, your Jupyter Notebook file would look very messy. **Note: Your submitted Jupyter Notebook file should contain all the output cells.** It is fine to implement the project on Google Colab, but you need to make sure that the outputs generated by Google Colab are saved in the Jupyter Notebook file that you submit for assessment.

7. Kaya users

This section is relevant for Kaya users only. Please see the file **Using-Kaya.pdf** released on LMS.

If you use Kaya interactively, you can use the command

```
ln -s /group/cits5017/shared/project2/temperatures-2026.pkl
```

to create a symbolic link to the data file in your directory.

To run batch jobs, you can modify the sample `proj2-2026.slurm` file supplied in the `/group/cits5017/shared/project2` directory.

8. Penalty on late submissions

See the URL below about late submission of assignments:

https://ipoint.uwa.edu.au/app/answers/detail/a_id/2711/~/-consequences-for-late-assignment-submission

Late submissions will be calculated at 5% of the total mark per day for the first 7 days (including weekends and public holidays) after which the project is not accepted.

9. Plagiarism

You should attempt the project by yourself. Collusion with (an)other student(s) is considered to be serious academic misconduct and can cause you to be suspended or expelled from the unit. Please see <https://www.uwa.edu.au/students/my-course/student-conduct> for more details.